F1 Info & Tickets - recommender system

Razvoj softvera II · Aida Ušanović (IB220012)

1. Purpose

The mobile app recommends **upcoming races** a customer is likely to want tickets for. The recommendations are shown on the mobile **Home** screen in the section *Recommended for you*, each with a short reason. They are served by `GET /api/Recommender` for the signed-in user.

2. Approach: content-based filtering

Every race is described by the properties of its circuit, and the user's taste is built from the races they interacted with. Upcoming races are ranked by how similar they are to that taste (cosine similarity), with a small boost for races that are popular with other fans.

Input	Source	Weight
Purchased tickets (not refunded)	`Tickets` table	1.0 per race
Race detail views	`RaceViews` table, written by `POST /api/Race/{id}/view` when the race details screen is opened	0.3 per view (at most 5 views per race)
Popularity	tickets sold per upcoming race, normalised to 0..1	20 % of the final score

3. Feature vector of a race

The feature space has one position per track type, per continent and per country (all read from the reference tables, so new values are included automatically). A race vector sets:

- **track type** of the circuit (Street / High-speed / Technical) = 1.0
- **continent** of the circuit = 1.0
- **country** of the circuit = 0.5 (a weaker hint than the type and the region)

4. Algorithm

1. Load the upcoming races, the user's purchased races and view counts, and ticket sales per race.
2. **User profile**: weighted average of the vectors of the races in the user's history

   ```
   weight(race) = 1.0 · purchased + 0.3 · min(views, 5)
   profile = Σ weight · vector(race) / Σ weight
   ```

3. **Candidates**: upcoming races the user has no ticket for, one per circuit (the nearest edition).
4. **Score** of each candidate:

```
similarity = cos(profile, vector(race)) = (p · r) / (|p| · |r|)
score = 0.8 · similarity + 0.2 · popularity
```

5. Sort by score (then by date) and return the top 5.
6. **Reason**: compares the race with the user's most frequent track type and continent, e.g. "*Similar to the races you follow: a "Street" circuit.*"
7. **Cold start**: a user without purchases or views gets the most popular upcoming races ("Popular with other fans").

Because views are recorded every time a race is opened, the recommendations change as the customer browses and buys tickets.

5. Main logic in the source code

Path: `F1Info&Tickets.Services/Services/RecommenderService.cs`, method `GetRecommendationsAsync` (API: `F1Info&Tickets.WebAPI/Controllers/RecommenderController.cs`).

```
28      public async Task<List<RaceResponse>> GetRecommendationsAsync(int userId, int topN = 5)
29      {
30          var now = DateTime.UtcNow;
31          var upcomingRaces = await _context.Races.IncludeRaceDetails()
32              .Where(r => r.Date > now)
33              .ToListAsync();
34
35          if (!upcomingRaces.Any())
36              return new List<RaceResponse>();
37
38          var purchasedRaceIds = await _context.Tickets
39              .Where(t => t.UserID == userId && t.Status != TicketStatus.Refunded)
40              .Select(t => t.RaceTicketCategory.RaceID)
41              .Distinct()
42              .ToListAsync();
43
44          var viewCounts = await _context.RaceViews
45              .Where(v => v.UserID == userId)
46              .GroupBy(v => v.RaceID)
47              .Select(g => new { RaceID = g.Key, Count = g.Count() })
48              .ToDictionaryAsync(x => x.RaceID, x => x.Count);
49
50          var historyIds = purchasedRaceIds.Union(viewCounts.Keys).ToList();
51          var historyRaces = await _context.Races.IncludeRaceDetails()
52              .Where(r => historyIds.Contains(r.RaceID))
53              .ToListAsync();
54
55          var popularity = await _context.Tickets
56              .Where(t => t.Status != TicketStatus.Refunded && t.RaceTicketCategory.Race.Date > now)
57              .GroupBy(t => t.RaceTicketCategory.RaceID)
58              .Select(g => new { RaceID = g.Key, Count = g.Count() })
59              .ToDictionaryAsync(x => x.RaceID, x => (double)x.Count);
60          var maxPopularity = popularity.Values.DefaultIfEmpty(0).Max();
61          double PopularityOf(Race r) => maxPopularity > 0 ? popularity.GetValueOrDefault(r.RaceID) / maxPopularity : 0;
62
63          // One entry per circuit (the nearest edition), skipping races the user already has tickets for.
64          var candidates = upcomingRaces
65              .Where(r => !purchasedRaceIds.Contains(r.RaceID))
66              .GroupBy(r => r.CircuitID)
67              .Select(g => g.OrderBy(r => r.Date).First())
68              .ToList();
69
70          if (!historyRaces.Any())
71          {
72              return candidates
73                  .OrderByDescending(PopularityOf)
74                  .ThenBy(r => r.Date)
75                  .Take(topN)
76                  .Select(r => ToResponse(r, "Popular with other fans (most tickets sold)."))
77                  .ToList();
78          }
```

GetRecommendationsAsync, part 1: user history, popularity, candidates and the cold-start fallback.

```
        var features = await BuildFeatureSpaceAsync();
        var userVector = new double[features.Count];
        double totalWeight = 0;
        foreach (var race in historyRaces)
        {
            var weight = (purchasedRaceIds.Contains(race.RaceID) ? PurchaseWeight : 0)
                + ViewWeight * Math.Min(viewCounts.GetValueOrDefault(race.RaceID), 5);
            var vector = CreateRaceVector(race, features);
            for (int i = 0; i < vector.Length; i++) userVector[i] += weight * vector[i];
            totalWeight += weight;
        }
        if (totalWeight > 0)
            for (int i = 0; i < userVector.Length; i++) userVector[i] /= totalWeight;

        var topTrackType = historyRaces.GroupBy(r => r.Circuit.TrackType.Name).OrderByDescending(g => g.Count()).First().Key;
        var topContinent = historyRaces.GroupBy(r => r.Circuit.City.Country.Continent.Name).OrderByDescending(g => g.Count()).First().Key;

        return candidates
            .Select(r => new
            {
                Race = r,
                Similarity = CosineSimilarity(userVector, CreateRaceVector(r, features)),
                Popularity = PopularityOf(r)
            })
            .OrderByDescending(x => SimilarityWeight * x.Similarity + PopularityWeight * x.Popularity)
            .ThenBy(x => x.Race.Date)
            .Take(topN)
            .Select(x => ToResponse(x.Race, BuildReason(x.Race, topTrackType, topContinent)))
            .ToList();
    }
```

GetRecommendationsAsync, part 2: weighted user profile, cosine similarity and the final score.

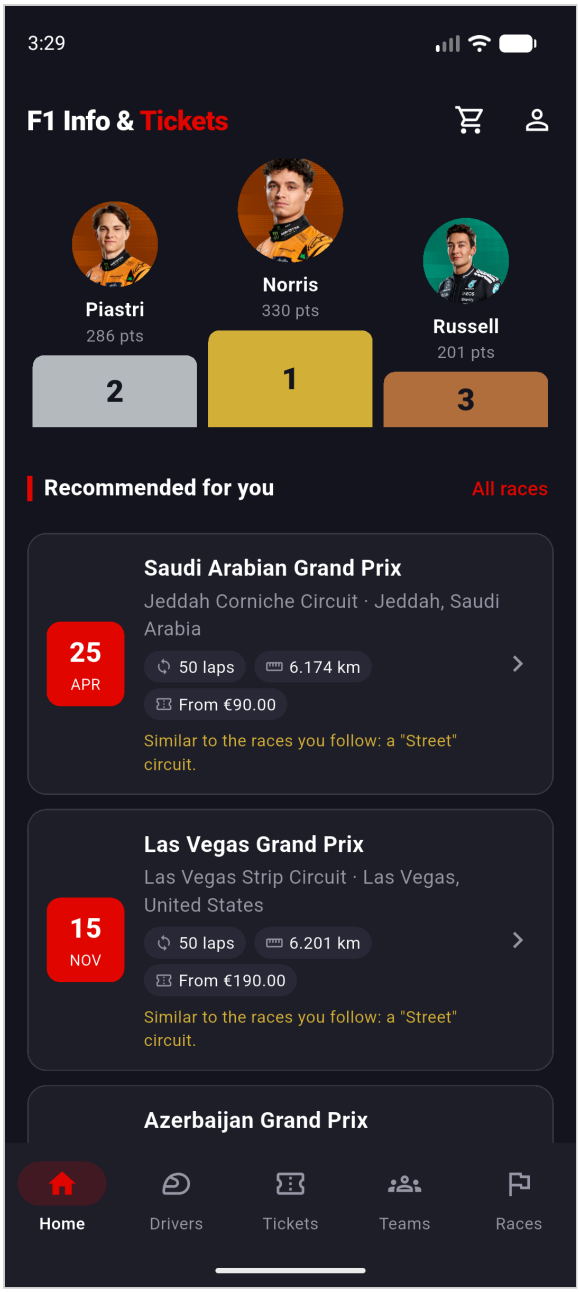
```
    // Track type and continent carry most of the signal; the exact country is a weaker hint.
    private static double[] CreateRaceVector(Race race, FeatureSpace features)
    {
        var vector = new double[features.Count];
        if (features.TrackTypes.TryGetValue(race.Circuit.TrackTypeID, out var t)) vector[t] = 1.0;
        if (features.Continents.TryGetValue(race.Circuit.City.Country.ContinentID, out var c)) vector[c] = 1.0;
        if (features.Countries.TryGetValue(race.Circuit.City.CountryID, out var k)) vector[k] = 0.5;
        return vector;
    }

    private static double CosineSimilarity(double[] a, double[] b)
    {
        double dot = 0, magA = 0, magB = 0;
        for (int i = 0; i < a.Length; i++)
        {
            dot += a[i] * b[i];
            magA += a[i] * a[i];
            magB += b[i] * b[i];
        }
        return magA == 0 || magB == 0 ? 0 : dot / (Math.Sqrt(magA) * Math.Sqrt(magB));
    }
```

CreateRaceVector and CosineSimilarity.

6. Recommendations in the running application

Path: `f1_info_and_tickets_frontend/lib/screens/mobile/home_tab.dart` (section *Recommended for you*; cards are rendered by `RaceCard` in `lib/widgets/mobile/mobile_widgets.dart`, data from `RaceProvider.getRecommendations()` in `lib/api/providers.dart`).



Mobile app, Home screen, signed in as `mobile`: recommended races with the reason under each race.